Integrating symbolic execution with LLMs for automated generation of C program specifications

Fanpeng Yang, Xu Ma, Shuling Wang*, Xiong Xu, Qinxiang Cao, Naijun Zhan, Xiaofeng Li, and Bin Gu*

***Abstract*—SESPEC generates verifier-checked C preconditions, postconditions, and loop invariants without a user-supplied verification goal. It combines symbolic execution, large language models (LLMs), and Frama-C/WP: on fully covered loop-free fragments, symbolic execution computes exact path-sensitive postconditions and composes ordinary non-recursive callee summaries; at loops, recursive structures, and other unsupported fragments, symbolic facts guide local LLM synthesis. Verifier feedback then directs refinement of failed candidates. We evaluate SESPEC on numerical, pointer, struct, and linked-list benchmarks and on the assertion-free SESPEC-400 benchmark. In the goal-independent study, the full pipeline achieves *Valid* rates of 68.0% with gpt-4o, 93.3% with gpt-5.4-mini, and 93.0% with gpt-5, exceeding the evaluated C/ACSL LLM baselines. These results show how path-sensitive symbolic context supports functional specification generation across both goal-dependent and goal-independent tasks.**

***Index Terms*—Program specification generation, loop invariants, symbolic execution, large language models, formal verification.**

I. INTRODUCTION

Legacy C programs often lack formal preconditions, postconditions, and loop invariants needed for program understanding, reuse, and verification. We study *goal-independent* functional specification generation: producing verifier-checked annotations that describe program behavior without a user-supplied assertion. Nested control flow, calls, pointers, structs, and inductive data make this task difficult because the generator must discover useful relations as well as prove them.

Existing work on program specification generation has largely focused on loop invariants. Traditional symbolic methods such as constraint solving [1], [2], Craig interpolation [3]–[5], recurrence analysis [6], [7], shape analysis [8], and dynamic methods such as Daikon [9] are each restricted to particular invariant forms or program classes; recent ML-augmented or SMT-augmented methods [10]–[12] are similarly confined to numeric programs. LLM-based approaches have begun to broaden the program coverage, but tend to produce specifications that are either ill-formed, too weak, or both, and recent integrations with static/formal techniques [13]–[16] only partially address this. AUTOSPEC [13] and Preguss [16] are goal-dependent (requiring a predefined verification target or RTE target) and degrade on complex data structures; SPECGEN [15] is goal-independent but only in the Java/JML setting, and "verifier-accepted" alone says nothing about strength. The remaining challenge is to generate informative, verifier-checked functional specifications for C programs with loops, calls, pointers, structs, and inductive data without a supplied goal.

We present SESPEC, which uses symbolic execution to drive LLM synthesis and formal verification. The analysis proceeds through the program and pauses at program boundaries (loops, calls, and recursive constructs), turning one whole-function specification task into a sequence of smaller local synthesis problems. On loop-free segments fully handled by the flattened-memory symbolic executor, symbolic execution computes an exact path-sensitive postcondition, which is the strongest postcondition within that model; outside this envelope, symbolic execution provides partial semantic anchors for LLM synthesis. At each boundary, the symbolic state, i.e. the symbolic store of variable values, the path conditions, and the types of the in-scope variables, collectively the *local symbolic context*, is injected into invariant templates and LLM prompts. Verifier feedback and symbolic information direct each repair of a failed candidate.

We implement SESPEC on top of QCP's symbolic execution engine [17], [18] and the Frama-C/WP verifier [19]. We evaluate SESPEC on benchmark families from prior invariant- and specification-generation work, covering numerical loops, multi-/nested loops, pointers, structs, and singly linked lists, together with an ablation study of each component. On numerical benchmarks, SESPEC approaches tools specialized for numerical programs and consistently outperforms recent LLM-based approaches such as AUTOSPEC [13] and ACInv [14]; on the pointer-, struct-, and list-heavy families that prior LLM-based tools handle poorly, it outperforms AUTOSPEC by a clear margin. On **SESPEC-400**, a 400-program benchmark with all assertions removed, SESPEC substantially outperforms single-shot LLM and AUTOSPEC baselines, and the specification comparison finds that its specifications provide more informative postconditions and stronger loop invariants. These results show that the symbolic-execution-guided design generalizes from the goal-dependent to the goal-independent setting without specialization.

In summary, the key contributions of this work are:

- **Symbolic-execution-guided specification synthesis**: A

*Corresponding authors: Shuling Wang (e-mail: wangsl@ios.ac.cn), Bin Gu (e-mail: gubin@ios.ac.cn).

F. Yang and X. Ma are with the Key Laboratory of System Software, Institute of Software, Chinese Academy of Sciences and also with the University of Chinese Academy of Sciences, Beijing, China (e-mail: yangfp@ios.ac.cn; maxu@ios.ac.cn

S. Wang and X. Xu are with the National Key Laboratory of Space Integrated Information Systems, Institute of Software, Chinese Academy of Sciences, Beijing, China (e-mail: wangsl@ios.ac.cn; xux@ios.ac.cn).

Q. Cao is with the School of Computer Science and School of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China (e-mail: caoqinxiang@sjtu.edu.cn).

N. Zhan is with the School of Computer Science, Peking University, Beijing, China (e-mail: znj@ios.ac.cn).

X. Li and B. Gu are with the Beijing Institute of Control Engineering, Beijing, China (e-mail: 1351127502@126.com; gubin@ios.ac.cn).

framework that drives LLM-based specification generation with path-sensitive symbolic states: exact semantic summaries on fully covered segments, and local symbolic context anchoring templates and prompts elsewhere.
- **Verifier-guided iterative refinement**: A multi-layer refinement mechanism that classifies Frama-C/WP failures and applies targeted repair strategies.
- **Goal-independent evaluation**: **SESPEC-400**, a benchmark of 400 C programs with assertions removed, used to study goal-independent specification generation across validity, strength, failure modes, and per-program cost.

The rest of the paper is organized as follows. Section II gives a motivating example and an overview; Sections III–IV present the core techniques of SESPEC; Sections V–VI report the empirical studies; Section VII surveys related work, and Section VIII concludes. System design, case studies, limitations, and threats to validity appear in Appendix E in the supplementary material, Appendix J in the supplementary material, Appendix K in the supplementary material, and Appendix L in the supplementary material, respectively.

II. A MOTIVATING EXAMPLE AND OUR APPROACH

Fig. 1 shows a fragment adapted from an industrial navigation component. Its `CheckCal` struct contains an array `pkv[10]`, its valid length `len`, and a sum field `chksum`. The caller `foo` invokes `CheckCalFun`, which sums the valid elements into `chksum`; an assertion in `foo` states the verification goal. This example raises three challenges:

- **Loop reasoning:** The summation loop `for(; i<pIp->len; i++)` requires determining whether the initial state allows loop entry and, if it executes, establishing invariants that relate the partial sum to the processed array prefix.
- **Structs and pointers:** The procedures read and write struct fields through pointers, so specifications must account for aliasing and side effects: memory safety for dereferencing `pIp->pkv[i]`, the write effect on `pIp->chksum`, and frame properties for unchanged fields of `pIp`.
- **Inductive predicates:** Aggregate array properties such as sums are most naturally expressed with inductive predicates or logic functions; writing them in a verifier-friendly form and proving the resulting annotations remains a central challenge.

Fig. 1 annotates the example with ACSL `requires`, `ensures`, `loop invariant`, and `assert` clauses. SESPEC generates the default preconditions, postconditions, and loop invariants; the assertion goal and additional `CheckCalFun` preconditions are supplied by the designers.

Before presenting our method, we attempted to generate the specification for this example using GPT-4o, Claude 3.7 Sonnet, and AUTOSPEC, given both the preconditions and the verification goal. The loop invariants they produced, shown below, were all found to be incorrect:

GPT-4O
```
loop invariant chksum == \sum(0, i-1,
  \lambda integer k; pIp->pkv[k]);
```
CLAUDE 3.7 SONNET
```
loop invariant chksum ==
  \sum(integer k = 0; k < i; pIp->pkv[k]);
```
AUTOSPEC
```
loop invariant chksum == \sum(0, i-1, pIp->pkv);
```

All three candidates include `0 <= i <= pIp->len` and a partial-sum relation, but omit the zero-iteration case (`pIp->len <= 0`); even a well-formed version therefore fails the base case. Explicitly prompting GPT-4o about this case did not produce a conditional invariant. Moreover, all three use `\sum` without defining an ACSL logic function; their lambda, quantified-binder, and raw-pointer variants are rejected by Frama-C/WP.

A. Our Approach

Fig. 2 shows an overview of our approach. Given a function and an optional verification goal, the objective is to automatically generate loop invariants and function pre-/postconditions. If a verification goal is provided, these specifications should be sufficient to prove it; otherwise, they should capture useful functional relations exposed by the available symbolic states. Following the three design choices of Section I, symbolic execution drives the pipeline and computes path-sensitive symbolic information at the current boundary; SESPEC injects these facts into invariant templates and ACSL skeletons, while a lightweight static route selects supporting examples and ACSL guidance. The LLM therefore receives both semantic facts from the current symbolic state and prompt support material matched to the program's syntactic features. The verifier then checks the candidate, and if it fails, the parsed verifier feedback is combined with the same local symbolic context to drive iterative refinement until a verifier-checked summary is produced. Each generation call is issued at a symbolic-execution boundary — a loop, an unspecified callee, a recursive data structure, or a recursive function — and once a verifier-checked summary is obtained, symbolic execution resumes from the same point using that summary.

Fig. 2 divides function-specification generation into four symbolic-execution-driven stages and an LLM fallback (Section III-D) for self-recursion, recursive data structures, external calls, path explosion, floating-point arithmetic, and runtime errors. The first stage, **Default Precondition Generation**, creates memory-safety conditions for symbolic execution (node ⓪ in Fig. 1); Section III-B gives the details.

From the default precondition, symbolic execution starts at function entry (node ① in Fig. 1). At a call (node ②), SESPEC pauses to recursively generate the callee specification (nodes ③–⑥), then applies that contract at the call site and resumes the caller. A call stack handles nested calls; generated specifications are stored for reuse, yielding bottom-up processing.

At a loop (node ⑤ in Fig. 1), symbolic execution pauses for **Loop Invariant Generation**. Loop Analysis uses the entry state and body to identify unchanged variables (`pIp->len`,

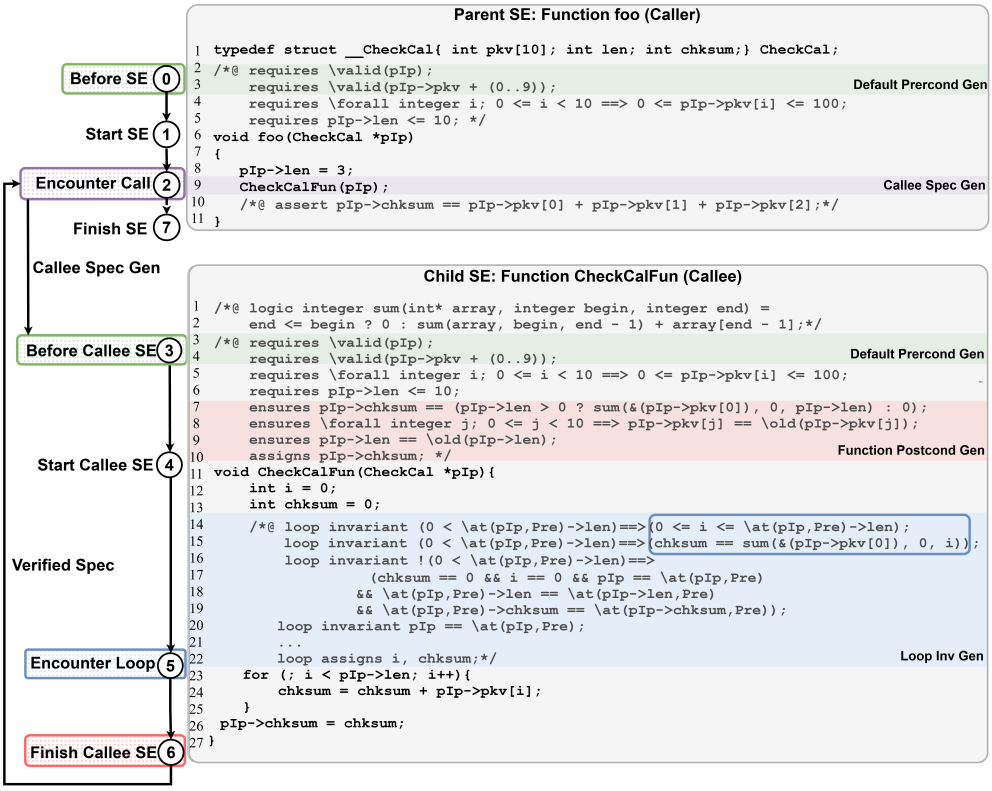


Fig. 1. Motivating Example

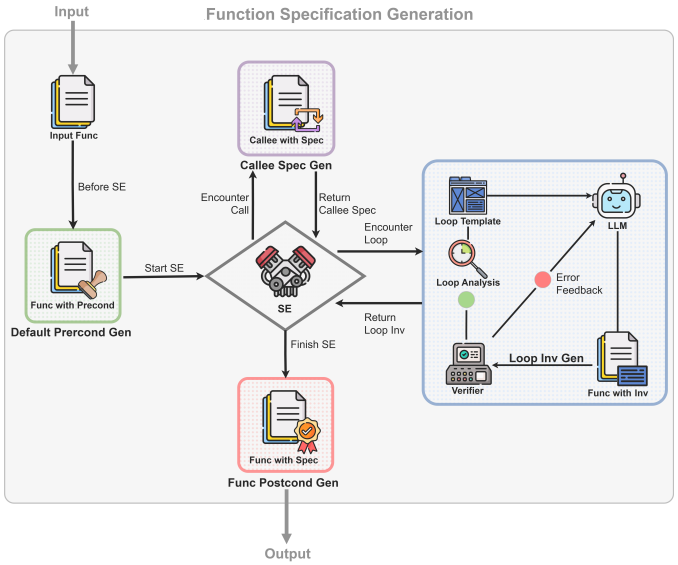


Fig. 2. Overview of Our Approach

`pIp`, `pIp->pkv`) and inductive variables (`i`, `chksum`). It records the guard `i < pIp->len` and the condition for at least one iteration, `0 < \at(pIp, Pre)->len`. These facts instantiate ACSL templates with guards, frame clauses, and relation placeholders, illustrated below (the complete annotations appear in Fig. 1).

```
loop invariant(0 < \at(pIp, Pre)->len)
  ==> PLACE_HOLDER_for_i;
loop invariant(0 < \at(pIp, Pre)->len)
  ==> PLACE_HOLDER_for_chksum;
loop invariant pIp == \at(pIp, Pre);
```

These skeletons provide specific `PLACE_HOLDER` tags for the LLM to populate (marked as the blue boxes in Fig. 1). Guided by these structured templates and symbolic execution results, the LLM produces loop invariants that capture useful relations among different types of variables. This symbolic-state-guided structure narrows the candidate space and makes key logical constraints explicit before the LLM fills the placeholders. For instance, the generated invariants correctly incorporate the loop-entry condition `(0 < \at(pIp, Pre)->len)`—the critical logical constraint whose omission caused the pure-LLM failures in Section II.

During Loop Invariant Generation, the verifier-guided refinement procedure is iteratively invoked to assess both the validity of the generated invariants and their usefulness for proving the given verification goal when such a goal is available. Depending on verifier feedback, different refinement strategies guide the LLM toward verifier-checked invariants.

When symbolic execution of the callee is completed (node ⑥ in Fig. 1), **Function Postcondition Generation** (marked in red in Fig. 2) is triggered. The resulting symbolic states are used to synthesize a postcondition that summarizes the callee's functional behavior. Upon completion, the generated pre-/postconditions are returned to the call site and reused as a functional summary in the caller (node ⑦ in Fig. 1), enabling symbolic execution of the caller to resume without re-analyzing the callee's internal control flow.

This example illustrates the full pipeline; goal-independent generation is evaluated separately in Section VI.

III. Pre-/Postcondition Generation

This section first introduces symbolic-execution-based specification generation for code segments without loops, and then presents the generation of general function specifications. Loop-invariant generation is deferred to the next section.

A. Symbolic execution and strongest postcondition

Before stating the proposition, we fix its assumptions.

At program point p, our *flattened memory model* starts from in-scope variables and parameters and enumerates a finite set $\mathcal{L}_p$ of atomic cells reached by pointer dereferences, struct-field selection, and element-wise traversal of statically known or parameter-bounded arrays. Each scalar or pointer cell receives a typed symbolic variable. Access paths resolved to the same cell share a location; pairs resolved as distinct are marked separated. Values, aliasing, and updates are therefore expressed over $\mathcal{L}_p$.

The model produces only finite, first-order facts over named cells; it does not use recursive or inductive predicates. These facts serve as preconditions and postconditions for two backends: QCP (in QCPL) for symbolic execution, and Frama-C/WP (in ACSL) for verification. What limits the expressiveness of the facts is Frama-C/WP's requirement for automatic SMT discharge: because the verification step must be fully automatic, the facts are confined to first-order and free of recursive/inductive predicates. Hence the model is deliberately finite. It handles scalars, structs, fixed arrays, and pointers with statically resolvable aliasing, but not unbounded or recursive structures (e.g., linked lists, trees), dynamic unbounded allocation, or accesses with a symbolic offset (e.g., pointer arithmetic). These cases fall outside the exact fragment and trigger the LLM fallback described in Section III-D.

Within this model, the symbolic execution of statements maintains a *symbolic state* $(PC_\pi, \sigma_{p,\pi})$ for each execution path π. At a program point p on path π, let $\mathcal{L}_p^\pi$ denote the set of symbolic locations currently tracked by the executor. This set includes the entry-point locations $\mathcal{L}_{p_0}$ reachable from the function parameters, together with the local variables that have entered scope along π before reaching p (hence $\mathcal{L}_{p_0} \subseteq \mathcal{L}_p^\pi$).

- PC_π denotes the path condition, i.e., a logical formula over function-entry symbolic values and constants that encodes the conditions needed to follow the path π.
- $\sigma_{p,\pi}$ denotes the symbolic store at program point p along path π, i.e., a mapping from each currently tracked symbolic location in $\mathcal{L}_p^\pi$ to a symbolic expression over the function-entry values. Since $\sigma_{p,\pi}$ is a mapping rather than a logical formula, we associate with it the store relation

$$\rho(\sigma_{p,\pi}) \equiv \bigwedge_{\ell \in \mathrm{dom}(\sigma_{p,\pi})} \ell^p = \sigma_{p,\pi}(\ell),$$

  where ℓ^p denotes the value of location ℓ at program point p.

The symbolic execution of a program is defined as a transformation over sets of symbolic states, branching at conditional control flow points. Formally, execution begins at the function entry point p_0 with the initial symbolic state (PC_0, σ_0), where σ_0 assigns an initial symbolic value to each location in the entry point location set $\mathcal{L}_{p_0}$, and PC_0 is the precondition constraining those initial values. For a program point p, let the set of all symbolic states reachable from p_0 be $\{(PC_{\pi_i}, \sigma_{p,\pi_i})\}_{i=1}^m$, where π_i ranges over all paths from p_0 to p. The reachable state pairs at p, projected to the tracked locations, satisfy

$$\bigvee_{i=1}^{m} \left(PC_{\pi_i} \wedge \rho(\sigma_{p,\pi_i})\right).$$

Here $\sigma_{p,\pi_i}(\ell)$ is an expression over the function-entry symbolic values, whereas ℓ^p denotes the corresponding value at program point p. An unchanged tracked cell retains the symbolic expression it had at the start of the current segment, and access paths known to alias refer to the same tracked location. Locations outside the finite tracked set are outside the exact claim. The path condition PC_{π_i} is expressed over the same function-entry symbolic values and includes the starting precondition. At the entry point p_0, the store relation is the identity relation, so the corresponding formula is PC_0 together with those identity equalities.

Given a segment $\mathcal{F}$, precondition P, and start point p_{start}, $\mathrm{SEStart}(\mathcal{F}, P, p_{start})$ explores feasible paths to the next boundary p (a loop entry, call, or exit of $\mathcal{F}$) and returns p. The traversed segment contains no loop or call, so its input/output behavior can be summarized exactly.

For a segment ending at p_{end}, let $\mathcal{L}_{p_{\mathrm{end}}}$ denote the observable endpoint locations tracked on every feasible path to p_{end}; path-specific locations are omitted.

We first define the notion of a *strongest postcondition* in the Hoare-logic sense for loop-free segments.

Definition 1. *(Strongest Postcondition) For a segment $\mathcal{F}$ and a precondition P, a formula Φ over the tracked initial and reached state variables is the strongest postcondition of $\mathcal{F}$ with respect to P, if*

- *$\{P\}\,\mathcal{F}\,\{\Phi\}$ holds, meaning every state pair (s_0, s) reachable by executing $\mathcal{F}$ from $s_0 \models P$ satisfies Φ.*
- *For any Ψ over the same tracked state-pair vocabulary such that $\{P\}\,\mathcal{F}\,\{\Psi\}$ holds, we have $\Phi \Rightarrow \Psi$ as a relation over state pairs.*

Let $\mathcal{F}$ be a program segment from p_{start} to p_{end} and P be a logical formula denoting the precondition. Write $\Pi = \mathit{Path}(p_{start}, p_{end})$ for the feasible paths explored under P, and $\Phi_{SP} = \mathit{SP}(\mathcal{F}, P, p_{start}, p_{end})$. We define

$$\Phi_{SP} = \bigvee_{\pi \in \Pi} \left(PC_\pi \wedge \rho(\sigma_{p_{end},\pi}|_{\mathcal{L}_{p_{end}}})\right).$$

Proposition 1 in Appendix A in the supplementary material states the precise guarantee, with its proof: under scope conditions (A1) — no external calls, no loops, flattened-model-representable accesses, and defined C semantics on every input satisfying P — and symbolic-execution exactness conditions (A2) — path completeness with exact path conditions and stores — Φ_{SP} is the strongest postcondition of $\mathcal{F}$ with respect to P.

Unsupported features cause *SP* to return *SymExecFailed* and invoke the LLM fallback; undefined behavior instead

produces an error. Fallback specifications may be verifier-checked but do not inherit the strongest-postcondition guarantee of Proposition 1 in Appendix A in the supplementary material.

B. Default Precondition Generation

$\mathrm{DefaultPre}(\mathcal{F})$ generates sufficient preconditions for the supported symbolic-execution model. By default, it requires the regions reached from distinct collected pointers to be disjoint and constrains array lengths and values to configured ranges. These restrictions define the input domain of the generated contract, rather than all well-defined C calls.

The generator processes the function parameters: for each pointer (including those inside structs and arrays), it emits a validity fact; after all pointers are collected, it appends a single separation fact asserting that the initial memory regions of these pointers are pairwise disjoint. Bound constraints restrict array lengths and integer values to the configured analysis ranges (e.g. $0 < arr_len < MAX_LEN$, $0 \le v \le MAX_VAL$ for integer value v). Validity and bounds help exclude common null-pointer and out-of-bounds runtime errors. QCP requires explicit initial memory predicates to justify dereferences, so the default separation fixes one layout rather than enumerating alias cases (Appendix B in the supplementary material). The default generator does not emit additional functional assumptions such as sortedness.

Designers may supply additional precondition clauses; these are conjoined with the auto-generated precondition into PC_0 at p_0 before symbolic execution begins. Fig. 1 illustrates both modes: the validity and separation facts on `CheckCalFun` are auto-generated, while the range constraints on its array values and length field are designer supplied.

In contrast, for recursive pointer-linked data structures (linked lists, trees, recursive structs), the number of reachable memory cells is determined only at runtime (by the list length or tree depth). Therefore, the static generator cannot emit a finite set of validity/separation facts for the entire shape; instead, the shape must be captured by an inductive predicate. In this case $\mathrm{DefaultPre}$ delegates the generation of the inductive shape predicate to the LLM fallback layer (Section III-D), which proposes the predicate and the corresponding precondition. The non-recursive part (validity of the head pointer and of primitive fields) is still emitted by the static generator. The LLM-generated predicate and precondition clauses are kept only if Frama-C/WP accepts them.

C. Function Specification Generation Algorithm

Algorithm 1 summarizes $\mathrm{FuncSpec}$: it combines symbolic execution with recursive callee-specification and loop-invariant generation, returning $\mathcal{F}$ with pre-/postconditions and loop invariants.

The algorithm works as follows. If $\mathcal{F}$ has already been annotated, it returns immediately (Lines 1–2); otherwise $\mathcal{F}$ is pushed onto the call stack *cS* — which records functions encountered during execution that still lack specifications — and a fresh loop stack *lS* is initialized for the nested loops of $\mathcal{F}$ (Lines 3–4). The default memory-safety precondition P is then computed by $\mathrm{DefaultPre}$ (Section III-B) at the entry

Algorithm 1: Function Specification Generation $\mathrm{FuncSpec}$

Input: function $\mathcal{F}$, call stack *cS*, annotated functions *aF*
Output: function $\mathcal{F}$ with specification $\mathcal{S}$

1 **if** $\mathcal{F} \in aF$ **then**
2 **return**
3 $cS.\mathrm{push}(\mathcal{F})$;
4 $lS \leftarrow \mathrm{InitLoopStack}()$;
5 $P \leftarrow \mathrm{DefaultPre}(\mathcal{F})$;
6 $p_{start} \leftarrow p_0$;
7 $p \leftarrow \mathrm{SEStart}(\mathcal{F}, P, p_{start})$;
8 **while** $p \neq p_{-1}$ **do**
9 $P \leftarrow SP(\mathcal{F}, P, p_{start}, p)$;
10 $p_{start} \leftarrow p$;
11 **if** $p = p_{loop}$ **then**
12 $\mathcal{F} \leftarrow \mathrm{LoopInvGen}(\mathcal{F}, p, lS, cS, aF, P, LLM)$;
13 **if** $p = p_{callee}$ **then**
14 $\mathcal{F}_{callee} \leftarrow \mathrm{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$;
15 $p \leftarrow \mathrm{SEStart}(\mathcal{F}, P, p_{start})$;
16 $\mathcal{S} \leftarrow SP(\mathcal{F}, P, p_{start}, p_{-1})$;
17 $\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{S})$;
18 $aF.\mathrm{append}(cS.\mathrm{pop}())$;
19 **return** $\mathcal{F}$;

point p_0, which assigns initial symbolic values to all atomic memory locations reachable from the variables in scope at p_0 (typically the function parameters), and symbolic execution begins from p_0 via $\mathrm{SEStart}$ (Lines 5–7); $\mathrm{SEStart}$ returns the next pause point p — the next loop entry, callee site, or function exit. While p has not reached the exit point p_{-1} (Line 8), the exact postcondition holding at p is aggregated through *SP* over the covered loop-free interval from the symbolic-execution results and p_{start} is reset to p (Lines 9–10), after which the dispatch is by boundary kind (Lines 11–15): a loop entry triggers $\mathrm{LoopInvGen}$, invoked with P as the loop precondition and p as the loop entry, returning $\mathcal{F}$ with the loop annotated by its invariant; a callee site triggers a recursive $\mathrm{FuncSpec}$ on the callee to obtain its specification; and in either case symbolic execution then resumes from the updated p_{start} under the freshly produced annotations. The two stacks together implement a compositional, bottom-up traversal: callee specifications are derived first and reused across calling contexts, and nested loops are processed before their enclosing loops. When symbolic execution reaches the function exit, *SP* aggregates the final postcondition $\mathcal{S}$, the annotated $\mathcal{F}$ is recorded in *aF*, and $\mathcal{F}$ is popped from *cS* and returned (Lines 16–19).

Two design points are worth making explicit. First, Algorithm 1 presents the idealized symbolic-execution-driven main path. In practice, the LLM-fallback layer described in Section III-D wraps this algorithm rather than being inlined into it. The fallback handles cases where symbolic execution cannot produce a usable summary (self-recursive functions, recursive data structures, external calls, *etc.*). Second, the loop-handling counterpart $\mathrm{LoopInvGen}$ (Algorithm 3 in Appendix D in the supplementary material), invoked at Line 12, is also guided by symbolic execution. It instantiates invariant templates using the loop precondition P, the symbolic store and path conditions at the loop entry, and the unchanged and non-inductive variable sets extracted by $\mathrm{LoopAnalysis}$ (Section IV-B). The LLM

then completes the template-specific invariant clauses, and the verifier-guided Refine step repairs them against Frama-C/WP feedback.

D. LLM Fallback

The LLM fallback wraps Algorithm 1 and has two triggers: (i) self-recursion or recursive data structures, routed at function entry because the executor neither computes recursive fixpoints nor enumerates unbounded shape cells; and (ii) *SP* returning *SymExecFailed* at function exit, covering external calls, path explosion, runtime errors, and other unsupported program features. Loops and ordinary callees instead use LoopInvGen and recursive FuncSpec; their verifier failures go to the separate refinement layer.

To replace a missing symbolic summary, the fallback LLM uses the code, available symbolic states and path conditions, prior callee and loop summaries, and an ACSL skeleton to propose preconditions, assigns clauses, postconditions, or inductive predicates. Prompt and task details appear in Appendix C in the supplementary material.

Fallback candidates are retained only if Frama-C/WP proves them under the configured memory model and solvers; failures enter refinement (Section IV-C). Accepted candidates can cover cases outside the exact symbolic-execution fragment as sound over-approximations, but lack the strongest-postcondition guarantee of Proposition 1 in Appendix A in the supplementary material.

IV. Loop Invariant Generation

The general loop invariant generation problem can be defined as follows:

Definition 2. *Given a loop l : `while B do C` of program $\mathcal{P}$, a precondition P that holds at entry l, and an (optional) postcondition Q, the loop invariant generation problem is to find a logical formula Inv that satisfies the following conditions:*

- ***Base:*** *$P \Rightarrow Inv$, i.e., the invariant Inv holds initially;*
- ***Preservation:*** *$\{Inv \wedge B\}\ C\ \{Inv\}$, i.e., if Inv holds at the start of a loop iteration and loop condition B is true, executing loop body C preserves Inv;*
- ***Termination:*** *$Inv \wedge \neg B \Rightarrow Q$, i.e., upon loop termination, B turns false, Inv remains true and implies postcondition Q.*

When Q is provided, the invariant must also prove that target; without Q, SESpec still seeks informative invariants. Symbolic states supply template guards, variable classifications, frame facts, and relation placeholders for LLM completion. Section V-E evaluates the resulting invariants.

A. Invariant Generation for Loops

Algorithm 3 (Appendix D in the supplementary material) takes a loop entry p_{loop}, its symbolic precondition P, and an LLM to synthesize an invariant; other inputs are as defined above.

The procedure has three steps. First, the LLM reads the outer loop body and answers guiding questions; this supports later synthesis but is never emitted as an annotation (Line 1). Second, LoopAnalysis processes inner loops and symbolically executes the body, returning unchanged variables *uV* and non-inductive variables *nV* while restoring the loop-stack depth (Lines 2–4). Third, LoopInvTemGen builds templates from that analysis, OuterLoopLLMCall fills them, and Refine repairs verification failures (Lines 5–9; Section IV-C).

LoopAnalysis does this by symbolically executing a single-iteration unfolding of the loop body and classifying *uV* and *nV* from the resulting postcondition; nested loops, for which no well-defined precondition exists at the enclosing loop boundary, are discharged bottom-up by direct LLM synthesis (NestedLoopLLMCall) without templates. The full procedure is given in Appendix D in the supplementary material (Algorithm 2).

B. Template Generation

LoopInvTemGen uses the loop body, precondition, and unchanged/non-inductive variable sets to build templates partly fixed by symbolic execution; OuterLoopLLMCall fills the remaining parameters (Algorithm 3 in Appendix D in the supplementary material). Prompt examples and ACSL guidance are described in Appendix E-B in the supplementary material. The six templates below cover four *SP*-derived variable forms, one form enabled by a supplied verification goal, and a free-form *sink*. The sink accommodates quantified array relations and inductive predicates that symbolic execution cannot anchor.

- Unchanged variables: For any variable $v \in uV$ and any execution path π_j to the loop entry p, v retains its symbolic value $\sigma_{p,\pi_j}(v)$ at the loop entry:
$$PC_{\pi_j} \Rightarrow v = \sigma_{p,\pi_j}(v)$$
- Loop skip conditional property: For a path π_j to p and a loop condition B, if B for path π_j is false, the loop is skipped without altering variables:
$$\neg(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow v = \sigma_{p,\pi_j}(v)$$
- Non-inductive variables: For a non-inductive variable v (whose value is updated independently of its value from the previous iteration) and any path π_j to p:
$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow (v = \sigma_{p,\pi_j}(v)) \vee \Phi(v)$$
where $\Phi(v)$ is the inductive invariant to be synthesized.
- Inductive variables: For an inductive variable v (which is neither unchanged nor non-inductive) and any path π_j to p:
$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(v)$$
- Verification goals: In addition to variable-specific templates, we also introduce templates corresponding to each verification goal. For a verification goal g and any path π_j to p:
$$(B \wedge (PC_{\pi_j} \wedge \rho(\sigma_{p,\pi_j}))) \Rightarrow \Phi(g)$$
- Free-form sink: an unanchored slot Φ that lets the LLM propose invariants involving quantifiers and inductive predicates that the deterministic forms above cannot anchor.

Symbolic templates turn invariant synthesis into constrained completion: they provide ACSL structure and path-sensitive facts while leaving relations for the LLM. Verifier-guided refinement then retains clauses accepted by Frama-C/WP.

C. Verifier-Guided Specification Refinement

Candidates may fail to parse, fail Frama-C/WP proof, or prove too weak to discharge a supplied assertion. Refinement takes the annotated program $\mathcal{F}$, editable annotation region $\mathcal{A}_0$, saved symbolic context Γ, target kind $k \in \{\textit{loop}, \textit{spec}\}$, and LLM; the rest of the program $\mathcal{F}^-$ stays fixed. It returns a verifier-checked $\mathcal{A}^*$ in $\mathcal{F}^*$, or the strongest verified subset found within the iteration budget.

Algorithm 4 (Appendix F in the supplementary material) verifies each candidate while editing only $\mathcal{A}$. Syntax/type errors trigger syntax repair; proof failures are classified by role and k, and only present categories enter the next prompt. Success stops the loop; budget exhaustion triggers conservative Houdini pruning.

Refinement has three layers:

Layer 1: syntax repair. For syntax or type errors, the LLM receives only the annotation region and verifier error; Frama-C/WP rechecks its repair before semantic proof.

Layer 2: strategy-driven semantic refinement or regeneration. For proof failures, SESPEC selects strategies for the observed obligation types (loop, function, callee, frame, or assertion; Table VII, Appendix F in the supplementary material). The LLM sees each failure with its matching strategy and revises invalid clauses, strengthens insufficient ones, or regenerates a globally inconsistent region.

Layer 3: Houdini pruning. At budget exhaustion, one Houdini pass removes verifier-identified failing clauses: loop invariants for loop refinement, or target `ensures` clauses and residual invariants for function refinement. It returns the strongest verifier-checked subset found.

The loop and function versions differ only in the error categories selected by $\mathrm{Classify}(\mathit{Errors}, k)$. The implementation also retains the best candidate by the lexicographic score

$$\mathrm{Score}(\mathcal{F}) = \big(\mathbf{1}_{\mathrm{parse}},\ \tfrac{|\mathrm{Valid}(VC)|}{|VC|},\ |C|\big),$$

Here $\mathbf{1}_{\mathrm{parse}}$ is one iff the candidate parses, VC is the candidate's verification conditions, $\mathrm{Valid}(VC)$ the proved subset, and C its retained clauses. The score prefers parseability, then proof ratio, then clause count. After two non-improving rounds (*patience* $= 2$), refinement resumes from the best candidate; Houdini also starts there at budget exhaustion.

V. EXPERIMENTS

This section evaluates SESPEC on benchmark families from prior invariant and specification generation work, which enables direct C/ACSL comparisons under established settings. Section VI complements this with a separate goal-independent study, where assertions are removed and the generator must decide what functional behavior to specify.

A. Experimental Setup

Benchmark: We evaluate our approach on benchmark families commonly used by recent specification and invariant generation tools. These include: **SyGuS** [20], collected by CODE2INV [10]; **OOPSLA** [21]; **NLA**, collected by LIPuS [11]; the datasets from **SV-COMP** and **Frama-C**, collected by AUTOSPEC [13]. In addition, we constructed three new benchmarks to capture more program scenarios: the **pIp** set consists of small C fragments adapted from a navigation component of an industrial control codebase, involving pointer and struct manipulations and branching; it provides a small, self-contained industrial sample of pointer and struct manipulation with branching, reformulated for Frama-C/QCP; the **list-S** set is derived from the LIG-MM [22] benchmark and includes all singly linked list programs from SLING and SV-COMP; and the **numer-S** set is a curated collection of additional programs, comprising 20 linear and 20 nonlinear loops, primarily drawn from CLAUSE2INV [12] and harder than those in **SyGuS** and **NLA**.

Verifier: All experiments are performed using Frama-C version 27.1 (Cobalt). We employ the WP plugin as the verification backend, which enforces a 10-second timeout per query, uses Z3 as the primary SMT solver, and applies the Typed memory model.

Model: We evaluate four LLMs of increasing capability: **GPT-3.5-turbo**, **GPT-4o-mini**, **GPT-4o**, and **Claude 3.7 Sonnet**. For all experiments, we use a fixed sampling temperature of 0.7 to balance output diversity and stability.

B. RQ1: Which program classes can SESPEC handle?

Table I classifies 395 programs by control flow and data type; blue cells need loop invariants, while red cells also need function contracts. SESPEC verifies 353/395 (89.4%), including pointer/struct and linked-list cases. It reaches about 96% on linear-loop and iteration-free programs but 77% on nonlinear loops, 63% on nested/multiple loops, and 0/3 on external/recursive calls. Thus lines of code alone poorly predict difficulty; the required invariant or summary matters more.

The weak classes have different causes. At nested or multiple loops, symbolic execution often lacks precise loop preconditions, leaving the LLM to infer relations across phases or loops. Nonlinear programs can require arithmetic facts, helper functions, or inductive definitions beyond the SMT budget. Recursive structures need inductive predicates beyond the finite memory model, while external and recursive calls lack an exact symbolic summary. The LLM fallback covers some simple recursive cases (Appendix J in the supplementary material); broader limits are discussed in Appendix K in the supplementary material.

> **Finding (RQ1).** SESPEC produces a *Valid* specification for 89.4% $(353/395)$ of programs spanning linear/non-linear loops, memory, and linked lists; the residual failures concentrate on reasoning-heavy cases involving nested/multi-loop relations, non-linear arithmetic, recursive data structures, or external/recursive calls outside the symbolic-execution fragment.

TABLE I
DATASET OVERVIEW BY PROGRAM TYPE.

Program Type	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S	#Program	#Solved
Linear Loop	133	30	14	0	20	7	0	0	204	196
Non Linear Loop	0	0	0	30	20	3	0	0	53	41
Nested & Multi Loop	0	16	7	0	0	1	0	0	24	15
Memory & Loop	0	0	2	0	0	17	12	24	53	45
Iteration Free	0	0	0	0	0	20	38	0	58	56
External & Recursive Call	0	0	0	0	0	3	0	0	3	0
#Program	133	46	21	30	40	51	50	24	395	
#Solved	132	40	16	25	34	39	44	23		353
Avg. LoC	19	24	22	21	19	18	57	24		

TABLE II
SPECIFICATION GENERATION PERFORMANCE UNDER PASS@1 AND PASS@5. THE COMPLETE PASS@1/3/5 TABLE IS GIVEN IN TABLE VIII (APPENDIX G IN THE SUPPLEMENTARY MATERIAL).

Benchmark	Model	Pass@1 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGuS	GPT-4o-mini	90.98	87.97	78.20	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	99.25	99.25	99.25
	Claude 3.7 Sonnet	100.00	100.00	99.25	100.00	100.00	99.25
OOPSLA	GPT-4o-mini	80.43	65.22	32.61	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	54.35	41.30	86.96	67.39	47.83
	GPT-4o	91.30	69.57	60.87	100.00	82.61	76.09
	Claude 3.7 Sonnet	93.48	69.67	63.04	97.83	93.57	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	76.19
	Claude 3.7 Sonnet	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67
	Claude 3.7 Sonnet	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 Sonnet	100.00	97.50	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	64.71	50.98	49.02
	GPT-3.5-turbo	77.08	60.42	52.08	83.33	62.50	54.17
	GPT-4o	82.35	72.55	66.67	86.27	78.43	72.55
	Claude 3.7 Sonnet	86.27	78.43	72.55	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	87.50	65.00	92.50	92.50	75.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	78.00
	Claude 3.7 Sonnet	100.00	88.00	82.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	91.67	87.50	79.17
	Claude 3.7 Sonnet	95.83	83.33	75.00	100.00	100.00	95.83

C. RQ2: What syntactic correctness, validity, and accuracy does SESPEC *achieve?*

We evaluate correctness through three metrics:

- **Syntax rate (Syn.):** the percentage of generated specifications syntactically accepted by the ACSL/Frama-C front end;
- **Validity rate (Val.):** the percentage of syntactically correct specifications whose generated clauses are accepted by Frama-C/WP under the verifier configuration in Section V. Pre-existing assertions or verification targets are excluded from Val.; generated loop invariants are valid only if they satisfy both the *Base* and *Preservation* conditions;
- **Accuracy rate (Acc.):** the percentage of valid specifications strong enough to prove the benchmark verification goal. For pre/postconditions, this means implying the given target assertions; for loop invariants, it further requires the *Termination* condition.

All metrics are assessed under Pass@k: a benchmark succeeds if any of up to k complete pipeline attempts (including repair, verification, and refinement; refinement iterations are not counted as separate attempts) satisfies the metric. The Pass@1 cost measurements for the goal-independent study are reported separately in Section VI-D.

Table II shows that syntax acceptance generally exceeds validity and accuracy, especially for weaker models on complex data. On list-S at Pass@1, GPT-3.5-turbo falls from 70.83% Syn. to 20.83% Val. and 8.33% Acc.: many linked-list annotations parse but fail verification or do not prove the goal. Claude 3.7 Sonnet reaches 95.83%/83.33%/75.00% on the same set.

Model gaps narrow on some families after multiple attempts. At Pass@5, GPT-4o and Claude reach the same Acc. on SV-COMP (76.19%) and NLA (86.67%); on numer-S they reach 82.50% and 85.00%. This pattern is consistent with symbolic guidance helping a less capable model recover useful candidates, although the table alone does not isolate that effect. Pass@1-to-Pass@5 gains show that extra attempts recover difficult cases, while Pass@3-to-Pass@5 gains are small on NLA, numer-S, and pIp (Appendix G in the supplementary material).

The lower Acc. on OOPSLA and SV-COMP is associated with nested or multiple loops, where symbolic execution may not derive a sufficiently strong loop precondition; OOPSLA also lacks explicit loop conditions in some programs. Frama-C cases additionally involve arrays, external calls, and recursion, which challenge the covered symbolic-execution fragment. These mechanisms explain why syntactically accepted annotations can still fail verification or leave a goal unproved.

Finding (RQ2). Syntactic correctness is generally high, but the gap between validity and accuracy widens with benchmark complexity and narrows with model capability; SESPEC narrows model gaps on several benchmark families and reaches most Pass@k gains by Pass@3, with most residual accuracy loss on nested/multi-loop and memory-heavy families.

D. RQ3: How much do symbolic execution and refinement each contribute?

We ablate 100 randomly sampled programs from Table I: 70 loop-only and 30 requiring full specifications. Configurations are token-matched, with at most three refinement iterations; Section VI separately tests the same components on 400 goal-independent programs without that compute cap. Table III compares the LLM alone (Pass@3), symbolic guidance without refinement (Pass@3), and the full pipeline (Pass@1).

TABLE III
ABLATION STUDY.

Method	Syn. (%)	Val. (%)	Acc. (%)
Loop invariant generation (70 programs)			
Only LLM	97.50	37.50	28.75
LLM + sym. exec.	95.71	62.86	60.00
LLM + sym. exec. + refine	95.71	95.71	87.21
Program specification generation (30 programs)			
Only LLM	66.67	26.67	16.67
LLM + sym. exec.	76.67	50.00	46.67
LLM + sym. exec. + refine	93.33	86.67	86.67

Table III shows high syntax rates but low validity and accuracy for the LLM alone. Symbolic guidance raises validity/accuracy by about 28 percentage points on average; refinement then raises loop-only validity to 95.71% and function-level validity from 50.00% to 86.67%. The latter is the largest gain and, as Section VI shows, the main cost driver.

> **Finding (RQ3).** Symbolic-execution guidance and verifier-guided refinement contribute distinct, complementary gains: over the LLM alone, symbolic execution adds ∼28 pp of validity/accuracy, and refinement then lifts validity to 95.7% on loop invariants and to 86.7% on full specifications.

E. RQ4: Does SESPEC outperform LLM- and SMT-based baselines?

Table IV compares each method's reported effectiveness under its original evaluation protocol: SESPEC permits five complete attempts, whereas AUTOSPEC repeats runs with internal synthesis/verification and ACInv uses augmentation cycles [13], [14]. The number and organization of synthesis and verification attempts therefore differ across methods.

Against numerical-invariant tools, SESPEC is on par with CLAUSE2INV on SyGuS, exceeds LIPuS but trails CLAUSE2INV by 6.6 pp on NLA, and reaches 87% on nested/multiple-loop OOPSLA. These tools specialize in numerical loops; SPECGEN is excluded from this C/ACSL table because it targets Java/JML.

For the C/ACSL LLM baselines, the matched-model comparisons show where the gains arise. On GPT-4o, SESPEC reaches 99.2% on SyGuS versus 86.5% for AUTOSPEC and 79.9% for ACInv; on OOPSLA it reaches 76.1%, matching ACInv and exceeding AUTOSPEC's 71.7%. SV-COMP remains difficult: SESPEC and ACInv both reach 76.2% on GPT-4o. SESPEC generally gains more from stronger models than AUTOSPEC, whose results vary little or decline on several families.

The largest differences occur on programs with pointers and data structures. On pIp, SESPEC reaches 88.0% with Claude 3.7 Sonnet versus AUTOSPEC's 4.0%; the latter often emits invalid ACSL, while symbolic-state-derived skeletons constrain both syntax and memory relations. On linked-list list-S, SESPEC reaches 95.8% with Claude, whereas AUTOSPEC remains at 0%. Inductive-predicate support also helps express properties beyond the finite symbolic-memory model in the Frama-C and list-S cases.

> **Finding (RQ4).** SESPEC matches tools specialized for numerical invariants and clearly outperforms LLM-based generators (AUTOSPEC, ACInv), with the largest margins on the pointer-, struct-, and linked-list families that prior LLM tools handle poorly; its accuracy also scales with model capability where AUTOSPEC's plateaus.

VI. GOAL-INDEPENDENT EVALUATION

We test goal-independent generation by removing all `assert` goals and comparing SESPEC with AUTOSPEC [13] (C/ACSL) and SPECGEN [15] (Java/JML).

A. Experimental Setup

We construct **SESPEC-400** from **FormalBench** [23] (250 Java programs), **SpecGenBench** (59 Java programs), and native-C **SV-COMP** (91). FormalBench originally contains OpenJML-verified Java programs with sequential, branching, and loop-based structures. We translate the Java programs to C while preserving control flow and data dependencies (Appendix H in the supplementary material), remove all `assert` statements, and use the Frama-C/WP protocol of Section V (Z3, Typed memory model).

All LLM methods use `gpt-4o`, `gpt-5-mini`, `gpt-5.4-mini`, and `gpt-5` at temperature 0.7 under Pass@1. Using one model family limits cross-family training and capability differences; the models in Section V were fixed by its baselines.

We compare full SESPEC (up to 10 refinement iterations), its *w/o refine* and *Pure LLM* (also without symbolic execution) ablations, and one-shot AUTOSPEC in C/ACSL. We run SPECGEN on the original Java programs in Java/JML as a cross-ecosystem reference. The ablations isolate symbolic guidance and refinement; only AUTOSPEC shares our verifier setting. Each run has a 1800-second outer timeout.

On all 400 programs, without the per-program compute cap of Section V, we measure validity, strength, cost, and failures. *Valid* means that the verifier discharges every obligation induced by the generated specification within the configured budget. The questions are:

- **RQ5:** How often does SESPEC produce a *Valid* specification?
- **RQ6:** How strong are SESPEC's specifications versus the baselines?
- **RQ7:** What does SESPEC cost in time, tokens, and API calls?
- **RQ8:** What are SESPEC's dominant failure modes?

TABLE IV
ACCURACY COMPARISON ACROSS BENCHMARKS AND BASELINES. SESPEC REPORTS PASS@5 (CF. TABLE II); BASELINES USE THEIR BEST REPORTED CONFIGURATION.

Method	SyGuS	OOPSLA	SV-COMP	NLA	numer-S	Frama-C	pIp	list-S
CODE2INV	54.9%	—	—	—	—	—	—	—
HOLA	—	**93.5%**	—	—	—	—	—	—
CLN2INV	93.2%	—	—	—	—	—	—	—
LIPuS	93.2%	—	—	83.3%	—	—	—	—
CLAUSE2INV	99.2%	—	—	**93.3%**	—	—	—	—
AUTOSPEC-3.5	85.7%	82.6%	**76.2%**	70.0%	75.0%	60.8%	14.0%	0.0%
AUTOSPEC-4o	86.5%	71.7%	66.7%	70.0%	72.5%	68.6%	22.0%	0.0%
AUTOSPEC-Claude 3.7	88.7%	80.4%	66.7%	60.0%	55.0%	74.5%	4.0%	0.0%
ACInv-3.5	71.4%	45.7%	42.9%	—	—	34.5%	—	—
ACInv-4o	79.9%	76.1%	**76.2%**	—	—	48.3%	—	—
SESPEC-3.5	93.2%	52.2%	66.7%	56.7%	67.5%	51.0%	76.0%	16.7%
SESPEC-4o	99.2%	76.1%	**76.2%**	86.7%	82.5%	72.6%	78.0%	79.2%
SESPEC-Claude 3.7	**100.0%**	87.0%	**76.2%**	86.7%	**85.0%**	**76.5%**	**88.0%**	**95.8%**

TABLE V
NATIVE-VERIFIER VALIDITY ON SESPEC-400.

Method	Model	Valid	Valid (%)
AUTOSPEC (C/ACSL)	gpt-4o	115	28.8
	gpt-5-mini	104	26.0
	gpt-5.4-mini	255	63.8
	gpt-5	288	72.0
SPECGEN (Java/JML)	gpt-4o	144	36.0
	gpt-5-mini	161	40.2
	gpt-5.4-mini	198	49.5
	gpt-5	251	62.7
SESPEC w/o refine w/o symbolic execution (Pure LLM)	gpt-4o	52	13.0
	gpt-5-mini	76	19.0
	gpt-5.4-mini	106	26.5
	gpt-5	55	13.8
SESPEC w/o refine	gpt-4o	126	31.5
	gpt-5-mini	107	26.8
	gpt-5.4-mini	132	33.0
	gpt-5	144	36.0
SESPEC	gpt-4o	272	68.0
	gpt-5-mini	333	83.3
	gpt-5.4-mini	373	93.3
	gpt-5	372	93.0

B. RQ5: How often does SESPEC *produce a* Valid *specification?*

Table V shows that full SESPEC has the highest C/ACSL *Valid* rate on every model: 68.0% on gpt-4o, 83.3% on gpt-5-mini, 93.3% on gpt-5.4-mini, and 93.0% on gpt-5. In that order, its gains are +55.0, +64.3, +66.8, and +79.2 pp over Pure LLM, and +39.2, +57.3, +29.5, and +21.0 pp over AUTOSPEC. Refinement alone adds +36.5, +56.5, +60.3, and +57.0 pp, at the cost of repeated LLM and verifier calls.

Without refinement, symbolic guidance raises *Valid* over Pure LLM on every model: from 13.0% to 31.5% on gpt-4o, 19.0% to 26.8% on gpt-5-mini, 26.5% to 33.0% on gpt-5.4-mini, and 13.8% to 36.0% on gpt-5. The unusually low gpt-5 Pure LLM rate is mainly syntactic: its uses of advanced ACSL behaviors and axiomatics often have malformed syntax, causing whole-file parse failures. Supplying local symbolic context before synthesis helps avoid these failures without repeated LLM sampling. For loop-free programs supported by the flattened memory model, symbolic execution can directly compute path-sensitive postconditions without an LLM call.

On `gpt-5.4-mini`, refinement gains saturate near $k = 5$; Houdini adds +14.1 pp, and differential prompts with rollback add about 23 pp over plain error-plus-code repair (Appendix F in the supplementary material, Fig. 7).

> **Finding (RQ5).** On **SESPEC-400** the full pipeline reaches 68–93% *Valid*; refinement is the dominant but most expensive source of gain, whereas symbolic-execution guidance is a cheaper one that needs no repeated LLM sampling.

C. RQ6: How strong are SESPEC*'s specifications versus the baselines?*

We compare specification strength along three dimensions: accepted input domains (preconditions), normal-return guarantees (postconditions), and predicates at corresponding loop points (loop invariants). All compared specifications have passed their respective verifier.

LLM judge and expert review. Separate judge calls compare preconditions by accepted-domain inclusion, postconditions by bidirectional implication over their common input domain, and invariants at corresponding loop points. Fig. 3 also combines the first two dimensions into a contract comparison. One side has a stronger contract when its precondition accepts at least as many inputs and its postcondition is at least as strong, with a strict advantage in at least one dimension; equality requires equality in both. Missing clauses mean `true`; unestablished implications count as Incomp. Expert review corrected 32 verdicts. Appendix I in the supplementary material reports these corrections, prompts, reviewer details, and an A/B-order audit.

Fig. 3 shows stronger SESPEC postconditions and loop invariants. Whereas AUTOSPEC accumulates sampled, verifier-accepted clauses, including redundant or tautological ones, SESPEC supplies local symbolic states to guide path-sensitive and inductive predicates.

AUTOSPEC accepts broader input domains more often (290 versus 96 pairs), while SESPEC has stronger postconditions (342 versus 18) and loop invariants (120 versus 52) more often. In the combined precondition–postcondition contract comparison, SESPEC leads in 172 pairs versus 64 for AUTOSPEC after expert review; WP finds 168 versus 58.

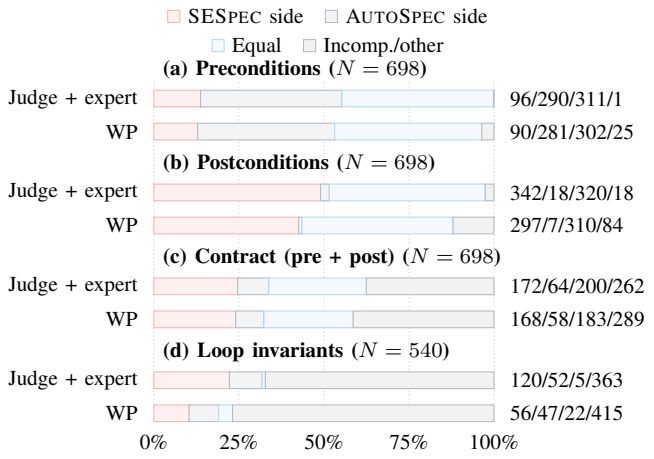


Fig. 3. Dimension-wise relations for AUTOSPEC versus SESPEC over identical data sets. A directional segment means a more permissive accepted domain for preconditions and a logically stronger predicate for postconditions or invariants. For contracts, a side leads when its precondition accepts at least as many inputs and its postcondition is at least as strong, with at least one strict advantage. Equal requires equality in both; conflicting or unresolved combinations enter Other. "Judge + expert" is the corrected judge result; WP checks the same implications. Right-hand counts follow the legend order.

Against SPECGEN (Java/JML), SESPEC leads on loop invariants (183 versus 9), while SPECGEN leads on postconditions (244 versus 100) and combined contracts (186 versus 92). The swapped-order contract counts are tied (129 versus 129); Appendix I in the supplementary material gives the full cross-language comparison.

Frama-C/WP cross-check. Separate Frama-C/WP harnesses cross-check 698 pre/postcondition pairs and 540 invariant pairs using Frama-C 27.1, Alt-Ergo and Z3, with five seconds per goal. In SESPEC/baseline/equal/incomparable order, the counts for preconditions, postconditions, and invariants are 90/281/302/25, 297/7/310/84, and 56/47/22/415, respectively.

WP assigns a direction only when it proves every corresponding obligation. An unproved implication enters Incomp whether it is false or true but beyond the prover; unsupported harnesses and unresolved loop correspondence also enter Incomp. Timeouts and automation limits therefore leave many invariants unordered. The judge followed by expert review can inspect semantic relations in these unresolved pairs, while WP provides an independent formal check. Fig. 3 confirms the precondition and postcondition trends, but WP orders fewer invariants.

Finding (RQ6). Symbolic-execution constraints yield safety-oriented preconditions that state validity, separation, and bound requirements; symbolic context guides stronger postconditions and loop invariants. This division explains the RQ6 pattern: AUTOSPEC more often accepts broader input domains, while SESPEC more often leads on guarantees and combined contracts. The cross-language comparison against SPECGEN (Java/JML) is reported in Appendix I in the supplementary material.

D. RQ7: What does SESPEC cost in time, tokens, API calls, and memory?

Costs are measured in the goal-independent evaluation, with all methods under Pass@1. Fig. 4 compares every attempt, including failures, on `gpt-5.4-mini`; the original AUTOSPEC time, call count, and validity labels are retained. Peak RSS stays below ~ 500 MB.

Each solid (hatched) bar segment sums valid-run (failed-run) costs over all attempts. Failed runs contribute 48.4 of SESPEC's 316.4 seconds per attempt, versus 523.3/933.8 for AUTOSPEC and 235.7/377.2 for SPECGEN. Time per obtained valid specification is 336.7, 1,465.3, and 756.3 seconds, respectively. SESPEC's 83.5k tokens per attempt include repeated refinement context without cache discounts.

Finding (RQ7). SESPEC's added framework complexity mainly appears as higher token use; its wall-clock time and API-call count are much closer to the baselines than the raw token totals alone suggest.

E. RQ8: What are SESPEC's dominant failure modes?

Fig. 5 analyzes an independent `gpt-5.4-mini` rerun with per-case logs. Capping refinement at three iterations instead of ten exposes more failures: 321 valid cases versus 373 in the main run. The panels show terminal outcomes and the dominant program feature in each of the 79 non-valid cases (42 *No valid*, 37 *No syntax*); for overlapping features, we choose the first explaining the blocking obligation. Table XIII (Appendix K in the supplementary material) separately classifies logged syntax and prover events by outcome and obligation type.

Loop invariant obligations account for $2{,}590/4{,}370 = 59.3\%$ of timeouts, especially establishment and preservation over arrays and heap arrays. Postcondition and `assigns` timeouts also occur.

Log outcomes *Timeout*, *Unknown*, and *Failed* denote timeouts, prover uncertainty, and backend or resource failures, not semantic counterexamples. They cannot separate incorrect specifications from correct but unproved obligations. Manual review more often found plausible properties limited by complex loop, array, or heap proofs, without ruling out synthesis errors. See Appendix K in the supplementary material for method and verifier limits.

Finding (RQ8). Residual failures frequently involve SMT timeouts on complex proof obligations, particularly those involving loop invariants, arrays, and heap-backed data. Manual inspection further identifies limitations in verifier automation as a major source of the remaining failures.

The ten cases in Appendix J in the supplementary material examine phase-split loop invariants, path-complete postconditions under C aliasing, linked-structure shape predicates, RTE safety versus functional specification, inductive helpers, signed arithmetic, recursion, SMT timeouts, and a 225-line industrial control pipeline (13/14 functions specified; 601/625 verification conditions discharged).

VII. RELATED WORK

Traditional approach. Dynamic methods infer candidate invariants from traces: Daikon [9] filters templates against observed tests, Sharma et al. [24] add SMT validation, and DIG [25] infers equations and polyhedra directly. Their results

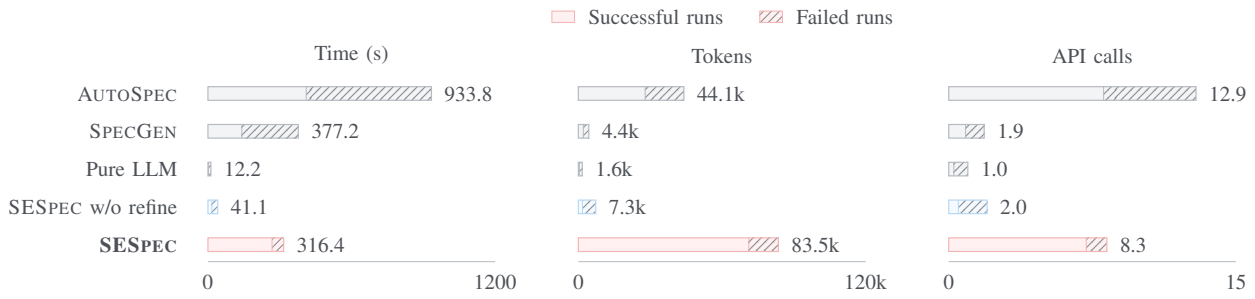
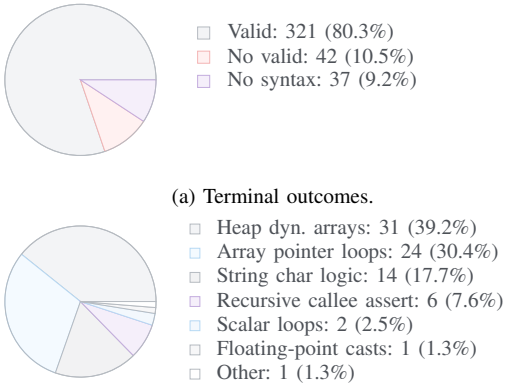


Fig. 4. Mean cost per attempted program with `gpt-5.4-mini`. Each bar stacks the valid-run (solid) and failed-run (hatched) contributions.



(a) Terminal outcomes.

(b) Root causes for non-valid cases.

Fig. 5. Failure outcome and root-cause summary for the `gpt-5.4-mini` SESPEC-400 rerun.

depend on test coverage. Static methods avoid this dependence through constraint solving [1], [2], abstract interpretation [26], [27], interpolation [3]–[5], recurrence analysis [28]–[30], and abductive or shape reasoning [21], [31], [32]. Their applicability often depends on the target property class or program form; SESPEC uses LLMs to cover the evaluated C/ACSL patterns.

These static lines make different trade-offs. Constraint methods can use Farkas' lemma and quantifier elimination to solve for invariant coefficients; polyhedral abstract interpretation uses widening to ensure convergence. Interpolation derives properties from reachability queries, while recurrence methods seek closed forms. Polynomial invariants have also been synthesized through semidefinite programming [33]. Such methods provide precise guarantees within their supported forms but may require templates or manual choices that limit broader applicability.

Machine learning and SMT-based approach. Learning-based invariant generators include CODE2INV [10], which combines graph representations with CEGIS [34] and reinforcement learning; LIPuS [11], which couples RL and SMT with a reward for sparse feedback; and CLN2INV [35], which uses continuous logic networks. Other learning approaches include [36]–[38]. CLAUSE2INV [12] combines LLM-proposed clauses with counterexamples; Loopy [39] filters LLM-generated ACSL invariants with Houdini and error-guided repair. These tools primarily infer goal-driven loop invariants, often for numeric programs, rather than goal-independent functional contracts. Table IV compares the available SyGuS/NLA/OOPSLA results; the numeric tools do not report OOPSLA results.

CODE2INV's reinforcement learning faces sparse rewards; LIPuS explicitly separates candidate generation from SMT checking and changes the reward structure to address that problem. CLAUSE2INV instead generates clauses and combines them using counterexamples, and Loopy moves this style of search to ACSL annotations checked by Frama-C. Despite their different search strategies, a supplied verification goal guides the invariant they seek. This matters for our setting, where the tool must choose functional properties to specify even when no assertion is present.

LLM-based approach. Pei *et al.* [40] fine-tune LLMs to predict invariants without formal guarantees. SPECGEN [15] generates Java/JML candidates and repairs failures through mutation. Preguss [16] uses abstract-interpretation RTE alarms to synthesize minimal safety specifications. For C/ACSL, AUTOSPEC [13] decomposes programs and prompts with examples, while ACInv [14] feeds loop and data-structure abstractions to the LLM. Section V compares SESPEC with AUTOSPEC and ACInv.

Their validation goals differ. Pei *et al.* assess predicted invariants without a proof obligation; SPECGEN seeks verifiable contracts but operates in Java/JML. Preguss focuses on preconditions sufficient for runtime-error freedom, whereas AUTOSPEC and ACInv target C/ACSL contracts using verifier feedback. These distinctions determine which results can be compared in the same verifier setting and which serve only as cross-language context.

ACInv and SESPEC both supply analysis facts to LLMs. ACInv uses loop and data-structure abstractions; SESPEC's symbolic snapshots additionally preserve path conditions, memory updates, alias distinctions, and field correlations needed for path-sensitive contracts. Preguss instead targets RTE freedom. SESPEC generates functional summaries and uses RTE reports as warnings or, when execution is blocked, as a reason to fall back; refinement may strengthen safety preconditions.

The distinction is especially relevant for branches and pointer updates: an abstract description of a loop or data structure may identify its shape without retaining the condition under which a particular post-state relation holds. Symbolic paths give the generator those conditional relations, while the fallback and verifier-guided refinement handle cases outside the exact symbolic fragment. The resulting advantage is local semantic context for synthesis; fallback and verifier-guided refinement remain necessary outside the exact symbolic fragment.

TABLE VI
POSITIONING OF SESPEC AGAINST REPRESENTATIVE SPECIFICATION AND INVARIANT GENERATION TOOLS.

Tool	Target	Goal	Method	Program Support	Array/Struct/Rec.	Validation
Daikon	invariants	test-driven	traces/templates	Java/C/C++ traces	support/no opt.	trace filter
LIPuS	loop inv.	goal-driven	SMT/RL clauses	C loops	unsupported	SMT
CLN2INV	loop inv.	goal-driven	continuous logic nets	C loops	unsupported	SMT
CLAUSE2INV	loop inv.	goal-driven	LLM clauses + CEX	C loops	unsupported	SMT
Loopy	loop inv.	goal-driven	LLM + Houdini repair	C loops	unsupported	Frama-C
AUTOSPEC	ACSL specs	goal-driven	decomposition + few-shot	C function/loops	support/no opt.	Frama-C
ACInv	ACSL specs	goal-driven	static abstractions	C function/loops	optimized	Frama-C
Preguss	ACSL specs	goal-driven	RTE alarms	C function/loops	support/no opt.	Frama-C
SPECGEN	JML specs	goal-independent	LLM + mutation	Java function/loops	support/no opt.	OpenJML
SESPEC	ACSL specs	goal-independent	symbolic states/templates	C function/loops	optimized	Frama-C

Lean-based specification generation. CLEVER [41] and VERINA [42] generate specifications and proofs for programs written in Lean. Lean supports expressive inductive predicates and proofs, but applying these systems to existing C code requires rewriting the implementation. SESPEC instead attaches ACSL contracts to native C.

Across these lines, goal-driven invariant tools target narrower properties; AUTOSPEC and ACInv use C/ACSL but depend on verification goals; SPECGEN is goal-independent in Java/JML; and Lean systems require a Lean implementation. SESPEC targets goal-independent, verifier-checked functional specifications for native C (Table VI).

VIII. CONCLUSION

SESPEC generates verifier-checked C specifications by grounding LLM synthesis in path-sensitive symbolic states and verifier feedback. On SESPEC-400, the full pipeline reaches *Valid* rates of 68.0% with gpt-4o, 93.3% with gpt-5.4-mini, and 93.0% with gpt-5. The comparison with AUTOSPEC also finds stronger postconditions and loop invariants on the both-valid C/ACSL pairs.

Inductive, recursive, and heap-heavy obligations account for many residual failures because they exceed the current Frama-C/WP automation budget. The analysis in Appendices K and L in the supplementary material identifies stronger proof support and broader symbolic-execution coverage as the next steps. Future systems can use an LLM agent to schedule symbolic execution, strengthen specifications, and introduce lemmas while continuing to ground each candidate in symbolic states and verifier feedback.

DATA AVAILABILITY

All datasets, implementation code, experimental results, and supplementary materials related to this work are publicly available at https://github.com/taylor-swift-13/SESpec.

REFERENCES

[1] M. Colón, S. Sankaranarayanan, and H. Sipma, "Linear invariant generation using non-linear constraint solving," in *Computer Aided Verification CAV*, ser. Lecture Notes in Computer Science, vol. 2725. Springer, 2003, pp. 420–432.

[2] H. Liu, H. Fu, Z. Yu, J. Song, and G. Li, "Scalable linear invariant generation with farkas' lemma," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA2, pp. 204–232, 2022.

[3] S.-W. Lin, J. Sun, H. Xiao, Y. Liu, D. Sanán, and H. Hansen, "FiB: Squeezing loop invariants by interpolation between forward/backward predicate transformers," in *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2017, pp. 793–803.

[4] T. Gan, L. Dai, B. Xia, N. Zhan, D. Kapur, and M. Chen, "Interpolant synthesis for quadratic polynomial inequalities and combination with EUF," in *Automated Reasoning - 8th International Joint Conference, IJCAR*, ser. Lecture Notes in Computer Science, vol. 9706. Springer, 2016, pp. 195–212.

[5] T. Gan, B. Xia, B. Xue, N. Zhan, and L. Dai, "Nonlinear Craig Interpolant Generation," in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 415–438.

[6] D. Beyer, M. Kettl, and T. Lemberger, "Decomposing software verification using distributed summary synthesis," *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1307–1329, 2024.

[7] D. Wonisch and H. Wehrheim, "Predicate analysis with block-abstraction memoization," in *International Conference on Formal Engineering Methods*. Springer, 2012, pp. 332–347.

[8] C. Calcagno, D. Distefano, P. O'Hearn, and H. Yang, "Compositional shape analysis by means of bi-abduction," in *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, 2009, pp. 289–300.

[9] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," *Sci. Comput. Program.*, vol. 69, no. 1, pp. 35–45, 2007, special issue on Experimental Software and Toolkits.

[10] X. Si, A. Naik, H. Dai, M. Naik, and L. Song, "Code2Inv: A Deep Learning Framework for Program Verification," in *Computer Aided Verification*. Springer International Publishing, 2020, pp. 151–164.

[11] S. Yu, T. Wang, and J. Wang, "Loop Invariant Inference through SMT Solving Enhanced Reinforcement Learning," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2023. Association for Computing Machinery, 2023, p. 175–187.

[12] W. Cao, G. Wu, T. Xu, Y. Yao, H. Wei, T. Chen, and X. Ma, "Clause2inv: A generate-combine-check framework for loop invariant inference," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, pp. 1009–1030, 2025.

[13] C. Wen, J. Cao, J. Su, Z. Xu, S. Qin, M. He, H. Li, S.-C. Cheung, and C. Tian, "Enchanting Program Specification Synthesis by Large Language Models Using Static Analysis and Program Verification," in *Computer Aided Verification: 36th International Conference, CAV 2024, Montreal*. Springer-Verlag, 2024, p. 302–328.

[14] R. Liu, G. Li, M. Chen, L.-I. Wu, and J. Ke, "Enhancing Automated Loop Invariant Generation for Complex Programs with Large Language Models," *CoRR*, vol. abs/2412.10483, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2412.10483

[15] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu, "SpecGen: Automated Generation of Formal Program Specifications via Large Language Models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, May 2025, pp. 666–666.

[16] Z. Wang, T. Lin, M. Chen, H. Li, M. Yang, X. Yi, S. Qin, Y. Luo, X. Li, B. Gu, L. Lu, and J. Yin, "A tale of 1001 loc: Potential runtime error-guided specification synthesis for verifying large-scale programs," *Proceedings of the ACM on Programming Languages*, vol. 10, no. OOPSLA1, pp. 1874–1902, Apr. 2026. [Online]. Available: http://dx.doi.org/10.1145/3798268

[17] L. Zhou, J. Qin, Q. Wang, A. W. Appel, and Q. Cao, "VST-A: A foundationally sound annotation verifier," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, 2024.

[18] X. Wu, Y. Feng, X. Lu, T. Lin, K. Liu, Z. Wang, S. Wu, L. Xie, C. Yang, H. Zhong, N. Zhan, Z. Hu, and Q. Cao, "Qcp: A practical separation logic-based c program verification tool," 2025. [Online]. Available: https://arxiv.org/abs/2505.12878

[19] P. Cuoq, F. Kirchner, N. Kosmatov, V. Prevosto, J. Signoles, and B. Yakobowski, "Frama-C: A software analysis perspective," in *International conference on software engineering and formal methods*. Springer, 2012, pp. 233–247.

[20] R. Alur, D. Fisman, S. Padhi, R. Singh, and A. Udupa, "Sygus-comp 2018: Results and analysis," *CoRR*, vol. abs/1904.07146, 2019.

[21] I. Dillig, T. Dillig, B. Li, and K. McMillan, "Inductive invariant generation via abductive inference," in *Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications*, ser. OOPSLA '13. Association for Computing Machinery, 2013, p. 443–456.

[22] C. Liu, X. Wu, Y. Feng, Q. Cao, and J. Yan, "Towards general loop invariant generation: a benchmark of programs with memory manipulation," *Advances in Neural Information Processing Systems*, vol. 37, pp. 129 120–129 145, 2024.

[23] T. Le-Cong, B. Le, and T. Murray, "Can llms reason about program semantics? a comprehensive evaluation of llms on formal specification inference," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 21 991–22 014.

[24] R. Sharma, S. Gupta, B. Hariharan, A. Aiken, P. Liang, and A. V. Nori, "A data driven approach for algebraic loop invariants," in *Programming Languages and Systems*. Springer Berlin Heidelberg, 2013, pp. 574–592.

[25] T. Nguyen, D. Kapur, W. Weimer, and S. Forrest, "DIG: A Dynamic Invariant Generator for Polynomial and Array Invariants," *ACM Trans. Softw. Eng. Methodol.*, vol. 23, no. 4, pp. 1–30, 2014.

[26] P. Cousot and R. Cousot, "Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '77. Association for Computing Machinery, 1977, p. 238–252.

[27] P. Cousot and N. Halbwachs, "Automatic discovery of linear restraints among variables of a program," in *Proceedings of the 5th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, ser. POPL '78. Association for Computing Machinery, 1978, p. 84–96.

[28] E. Rodríguez-Carbonell and D. Kapur, "Automatic generation of polynomial loop invariants: Algebraic foundations," in *International symposium on Symbolic and algebraic computation (ISSAC)*. Springer, 2004, pp. 266–273.

[29] ——, "Generating all polynomial invariants in simple loops," *J. Symb. Comput.*, vol. 42, no. 4, pp. 443–476, 2007.

[30] J. Cyphert and Z. Kincaid, "Solvable polynomial ideals: The ideal reflection for program analysis," *Proc. ACM Program. Lang.*, vol. 8, no. POPL, pp. 724–752, 2024.

[31] M. Chalupa and J. Strejček, "Backward Symbolic Execution with Loop Folding," in *Static Analysis*. Springer International Publishing, 2021, pp. 49–76.

[32] T. Nguyen, M. B. Dwyer, and W. Visser, "SymInfer: inferring program invariants using symbolic states," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '17. IEEE Press, 2017, p. 804–814.

[33] H. Wu, Q. Wang, B. Xue, N. Zhan, L. Zhi, and Z. Yang, "Synthesizing Invariants for Polynomial Programs by Semidefinite Programming," *ACM Trans. Program. Lang. Syst.*, vol. 47, no. 1, pp. 1 – 35, Feb. 2025.

[34] A. Abate, C. David, P. Kesseli, D. Kroening, and E. Polgreen, "Counterexample Guided Inductive Synthesis Modulo Theories," in *Computer Aided Verification*. Springer International Publishing, 2018, pp. 270–288.

[35] G. Ryan, J. Wong, J. Yao, R. Gu, and S. Jana, "CLN2INV: learning loop invariants with continuous logic networks," in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020.

[36] S. Chakraborty, S. Lahiri, S. Fakhoury, A. Lal, M. Musuvathi, A. Rastogi, A. Senthilnathan, R. Sharma, and N. Swamy, "Ranking LLM-generated loop invariants for program verification," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 2023, pp. 9164–9175. [Online]. Available: https://aclanthology.org/2023.findings-emnlp.614/

[37] H. Wu, C. Barrett, and N. Narodytska, "Lemur: Integrating Large Language Models in Automated Program Verification," in *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024.

[38] M. A. A. Pirzada, G. Reger, A. Bhayat, and L. C. Cordeiro, "LLM-Generated Invariants for Bounded Model Checking Without Loop Unrolling," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. Association for Computing Machinery, 2024, p. 1395–1407.

[39] A. Kamath, A. Senthilnathan, S. Chakraborty, P. Deligiannis, S. K. Lahiri, A. Lal, A. Rastogi, S. Roy, and R. Sharma, "Finding inductive loop invariants using large language models," 2023. [Online]. Available: https://arxiv.org/abs/2311.07948

[40] K. Pei, D. Bieber, K. Shi, C. Sutton, and P. Yin, "Can large language models reason about program invariants?" in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023, pp. 27 496 – 27 520.

[41] A. Thakur, J. Lee, G. Tsoukalas *et al.*, "CLEVER: A curated benchmark for formally verified code generation," in *Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2025.

[42] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song, "VERINA: Benchmarking verifiable code generation," in *The Fourteenth International Conference on Learning Representations*, 2026.

[43] C. Cadar, D. Dunbar, and D. R. Engler, "KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs," in *8th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2008, pp. 209–224.

[44] V. Kuznetsov, J. Kinder, S. Bucur, and G. Candea, "Efficient state merging in symbolic execution," in *Proceedings of the 33rd ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI '12. New York, NY, USA: Association for Computing Machinery, 2012, pp. 193–204. [Online]. Available: https://doi.org/10.1145/2254064.2254088

[45] T. Avgerinos, A. Rebert, S. K. Cha, and D. Brumley, "Enhancing symbolic execution with veritesting," in *Proceedings of the 36th International Conference on Software Engineering*, ser. ICSE 2014. New York, NY, USA: Association for Computing Machinery, 2014, pp. 1083–1094. [Online]. Available: https://doi.org/10.1145/2568225.2568293

[46] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu, "How powerful are LLMs in generating formal program specifications?" in *Forty-third International Conference on Machine Learning*, 2026.

[47] Z. Li, Z. Yang, D. He, H. Zhao, A. Zhao, S. Tang, K. Yang, A. Gupta, Z. Su, and C. Jin, "Goedel-code-prover: Hierarchical proof search for open state-of-the-art code verification," 2026. [Online]. Available: https://arxiv.org/abs/2603.19329